

Lecture 2 Bits, Bytes, and Integers

Encoding Byte Values

- Byte = 8 bits
- Binary, decimal and hexadecimal transferring

Byte-Oriented Memory Organization

Programs refer to virtual addresses, conceptually very large array of bytes.

Every byte of memory is identified by a unique number, known as its address.

The set of all possible addresses is known as virtual address space.

Machine Words

Previous machines use 32-bit words while modern systems use 64-bit words. The machine words determine the address space (2^{64}).

For a machine with w -bit word size, its virtual addresses $\in [0, 2^w - 1]$.

Data Representation

C declaration	32-bit	64-bit
char	1	1
short int	2	2
int	4	4
long int	4	8
long long int	8	8
char *	4	8
float	4	4
double	8	8

Byte Ordering

A multi-byte is stored as a **contiguous** sequence of bytes with the address of the object given by the **smallest address** of the bytes used. For example, a 4-byte variable x has address 0x100, i.e., $\&x$ is 0x100, then it stores in 0x100, 0x101, 0x102, 0x103.

It can be divided into big endian and little endian.

Big endian (Sun, IBM):

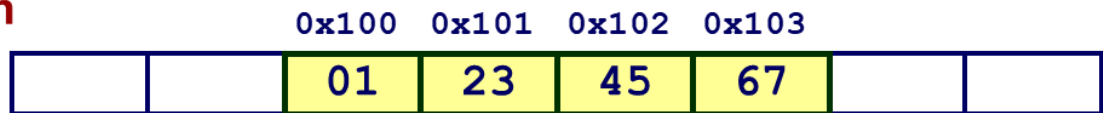
Least significant byte has highest address

Little endian (x86):

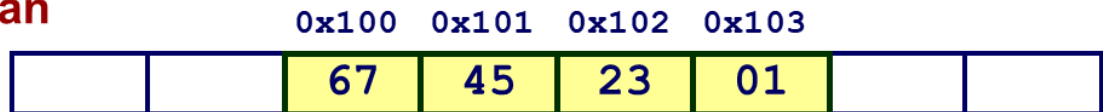
Least significant byte has lowest address

For example, x is 0x01234567, $\&x$ is 0x100,

Big Endian



Little Endian



Representing Strings

Strings in C are represented by **array of characters** and each character encoded in ASCII format(standard 7-bit encoding, "0" has code 0x30 and digit i has code $0x30 + i$) or Unicode.

A string should be null-terminated. Final character is zero (0x0, not character "0" 0x30).

Byte ordering **does not affect compatibility**.

Boolean Algebra

- Four basic operations: AND(&), OR(|), NOT(~), XOR(^)
- General Boolean algebra: Operate bitwise on bit vectors.

Representation and Manipulating Sets

- Representation: Width w bit vector represent subset of $\{0, \dots, w - 1\}$ for $a_i = 1$ if $i \in A$. For example, $\{0, 3, 5, 6\}$ can be expressed by a bit vector 01101001 which is 1 at 6,5,3,0-th bit. (76543210)

- Operations: AND is intersection of sets, OR is union of two sets, XOR is symmetric difference of two sets (union minus intersection, $(A - B) \cup (B - A)$), NOT is complement of a set.

Shift Operations

- Left shift: filling the right end with k zeros.
- Right shift
 - Logical shift: fill with 0's on the left
 - Arithmetic shift: Replicate most significant bit on left.

Classwork

2.5, 2.8, 2.16

Encoding Integers

Unsigned:

$$B2U_w(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

Two's complement:

$$B2T_w(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

For 2's complement, most significant bit indicates sign 0 for nonnegative, 1 for negative.

For unsigned values, $n \in [0, 2^w - 1]$, for two's complement values, $n \in [-2^{w-1}, 2^{w-1} - 1]$.

And we can observe that $|Tmin| = Tmax + 1$, $Umax = 2 * Tmax + 1$.

Mapping Between Signed & Unsigned

Keep **bit representation and reinterpret**. (T2B, B2U and U2B, B2T)

(Note that T2B is that first judge the most significant bit, then use the number minus $-x_{w-1} \cdot 2^{w-1}$ and then transfer the result)

or use the function:

$$T2U_w(x) = \begin{cases} x + 2^w, & x < 0 \\ x, & x \geq 0 \end{cases}$$

$$U2T_w(u) = \begin{cases} u, & u < 2^{w-1} \\ u - 2^w, & u \geq 2^{w-1} \end{cases}$$

In C, constants by default are considered to be signed integers (unsigned if have "U" as suffix), casting between signed and unsigned is same as U2T and T2U.

Sign Extension

Task:

- Given w -bit signed integer x .
- Convert it into $w + k$ -bit integer with same value.

Rule:

- Make k copies of sign bit.
- $x' = [x_{w-1}, \dots, x_{w-1}, x_{w-2}, \dots, 0]$

Truncating: Basic Rules

Directly truncate of the bit representation. (Unsigned: mod operation; signed: similar to mod)

For addition or multiplications that produces the extra digits than w , directly truncate the extra digit in binary representation and use B2T or B2U to rest digits.

Unsigned Addition

$$s = UAdd_w(u, v) = (u + v) \bmod 2^w$$

That is,

$$UAdd_w(u, v) = \begin{cases} u + v, & u + v < 2^w \\ u + v - 2^w, & u + v \geq 2^w \end{cases}$$

Two's Complement Addition

$$TAdd_w(u, v) = \begin{cases} u + v + 2^w, & u + v < TMin_w \text{ (Negative overflow)} \\ u + v, & TMin_w \leq u + v \leq TMax_w \\ u + v - 2^w, & u + v > TMax_w \text{ (Positive overflow)} \end{cases}$$

Multiple

$$UMult_w(u, v) = (u \cdot v) \bmod 2^w$$

$$TMult_w(u, v) = U2T_w((u \cdot v) \bmod 2^w)$$

Power-of-2 Multiply with Shift

Operations by $u \ll k$ gives $u * 2^k$. (Both signed and unsigned)

For other multiplication, it can be represented by shift and addition.

Power-of-2 Divide with Shift

$u \gg k$ gives $\lfloor u/2^k \rfloor$. Note that unsigned uses logical shift and signed uses arithmetic shift (rounds wrong when it is positive).

Classwork

Problem 2.17 & 2.40